2.4. Conversiones entre distintos tipos de datos (Criterio 2.d)

- **Resultado de Aprendizaje 2 (RA2):** *Escribe sentencias simples, aplicando la sintaxis del lenguaje y verificando su ejecución sobre navegadores Web.*

- **Criterio Curricular Oficial: CE 2.d** (S*e han reconocido y comprobado las peculiaridades del lenguaje respecto a las conversiones entre distintos tipos de datos*).

- **Ponderación del Criterio:** (**8% del RA2** | **0,4% sobre la calificación final del módulo**).

A. El Sistema de Tipado Débil y la Coerción de Tipos

En JavaScript, las variables no tienen tipo fijo asignado en la declaración, sino que los tipos están asociados a los valores en tiempo de ejecución. Al ser un lenguaje **débilmente tipado**, el motor intenta resolver operaciones entre tipos heterogéneos mediante la **coerción de tipos** (*Type Coercion*), es decir, la transformación de un tipo de dato en otro según el contexto semántico de la instrucción.

Existen dos modalidades de conversión:

1. **Conversión automática o implícita:** La realiza el intérprete siguiendo reglas internas de conveniencia.
2. **Conversión manual o explícita:** La fuerza el programador mediante funciones del estándar (Number(), String(), etc.).

B. Conversión Automática de Tipos (Coerción Implícita)

1. Coerción a Cadenas de Texto (*String Coercion*)

Ocurre fundamentalmente con el operador binario de suma (+). Si uno de los operandos es de tipo texto, JavaScript convierte automáticamente el otro operando a cadena y realiza una concatenación:

```
// El número 5 se transforma a cadena "5" y se unen:
let resultado1 = 12 + "5";
console.log(resultado1); // Imprime: "125" (tipo string)

let resultado2 = "67" + 11;
console.log(resultado2); // Imprime: "6711" (tipo string)

// Comportamiento con booleanos y nulos en cadenas:
console.log("Acceso: " + true); // "Acceso: true"
console.log("Dato: " + null);    // "Dato: null"
```

2. Coerción a Números (*Numeric Coercion*)

Aparece cuando se aplican operadores estrictamente matemáticos (-, *, /, %, **). En estos contextos, el motor fuerza a que los operandos se transformen a números:

```
// La resta no admite concatenación, por tanto fuerza la resta
numérica:
console.log("10" - 2); // 8 (el texto "10" se convierte en número
10)

// La multiplicación convierte ambas partes a número:
console.log(12 * "5");  // 60
console.log("67" * 11); // 737

// Coerción de valores lógicos:
// true se asume como 1; false se asume como 0
console.log(true * 7); // 7 (1 * 7)
console.log(9 - false); // 9 (9 - 0)

// Coerción de null y undefined:
// null representa vacío intencionado y se evalúa como 0
console.log(null * 6); // 0 (0 * 6)

// undefined no tiene representación numérica coherente, por lo
que genera NaN
console.log(undefined / 7); // NaN (Not-a-Number)

// Operaciones imposibles:
console.log("texto" * 8);       // NaN (no se puede interpretar
"texto" como número)
console.log(Infinity - "texto"); // NaN
console.log(NaN + 4);           // NaN
```

3. Coerción a Booleanos (*Boolean Coercion*)

Sucede de forma automática dentro de estructuras condicionales (if, bucles while) o cuando intervienen operadores lógicos (&&, ||, !). El motor clasifica el dato según sea un valor **Falsy** o **Truthy**:

REGLA MNEMOTÉCNICA: Los 8 valores Falsy en JavaScript
Cualquier dato fuera de esta lista cerrada se evalúa como true:
false, 0, -0 , 0n, "" (cadena vacía), null, undefined y NaN

C. Conversión Manual de Tipos (Conversión Explícita)

Delegar en la conversión automática puede introducir errores de cálculo o desbordamientos difíciles de rastrear en producción. La práctica recomendada consiste en forzar el tipo explícitamente.

1. Transformación Explícita a Número

Existen tres formas estándar en el lenguaje:

- **Función constructora Number(valor):** Intenta convertir el valor completo a un número. Si contiene caracteres no numéricos, devuelve de inmediato NaN:

```
console.log(Number("42"));     // 42
console.log(Number("42px"));   // NaN (falla por las letras
'px')
console.log(Number(true));     // 1
console.log(Number(false));    // 0
console.log(Number(null));     // 0
console.log(Number(undefined));// NaN
```

- **Funciones de análisis sintáctico parseInt() y parseFloat():** Leen de izquierda a derecha extrayendo la parte numérica hasta encontrar un carácter no válido:

```
console.log(parseInt("42px", 10));    // 42 (base 10
especificada)
console.log(parseFloat("19.95 euros")); // 19.95
console.log(parseInt("ancho: 100"));  // NaN (comienza por
una letra)
```

- **Operador unario positivo (+):** Es la vía más corta y eficiente en JavaScript para convertir una cadena a número:

```
let entrada = "150";
let total = +entrada; // Convierte "150" a 150 numérico
console.log(typeof total); // "number"
```

2. Transformación Explícita a Cadena de Texto

- **Función constructora String(valor):** Funciona con cualquier tipo de dato, incluso con valores primitivos nulos o indefinidos:

```
console.log(String(123));       // "123"
console.log(String(true));      // "true"
console.log(String(null));      // "null"
console.log(String(undefined)); // "undefined"
```

- **Método valor.toString():** Se invoca sobre el valor, pero **provoca un error bloqueante TypeError** si la variable contiene null o undefined:

```
let edad = 20;
console.log(edad.toString()); // "20"

let vacio = null;
// vacio.toString(); // ERROR: Uncaught TypeError: Cannot
read properties of null
```

3. Transformación Explícita a Booleano

- **Función Boolean(valor):** Evalúa directamente según la regla de valores falsy/truthy:

```
console.log(Boolean(1));        // true
console.log(Boolean(0));        // false
console.log(Boolean(""));       // false
console.log(Boolean(" "));      // true (un espacio en blanco
cuenta como texto)
console.log(Boolean([]));       // true (los objetos y
matrices son siempre truthy)
```

- **Operador de doble negación (!!):** Invierte dos veces el valor, forzándolo a booleano de forma concisa:

```
let token = "XYZ-123";
let tieneSesion = !!token; // Convierte a true si hay cadena,
false si está vacía
console.log(tieneSesion);  // true
```

!!valor es exactamente lo mismo que Boolean(valor) en cuanto a su resultado final. Ambos métodos fuerzan a JavaScript a convertir cualquier dato en un booleano puro (true o false).

D. Tabla Resumen de Coerciones y Casos Especiales

La siguiente matriz ilustra las peculiaridades de conversión que el estudiante y futuro técnico debe conocer para superar evaluaciones técnicas y evitar errores en código:

Valor Original	Convertido a Number(x)	Convertido a String(x)	Convertido a Boolean(x)
0	0	"0"	false
1	1	"1"	true
"" (cadena vacía)	0	""	false

"0" (texto con cero)	0	"0"	**true** (¡es un texto no vacío!)
"false"	NaN	"false"	**true**
true	1	"true"	true
false	0	"false"	false
null	**0**	"null"	false
undefined	**NaN**	"undefined"	false
NaN	NaN	"NaN"	false
[] (matriz vacía)	0	""	**true**
[25]	25	"25"	**true**
{} (objeto vacío)	NaN	"[object Object]"	**true**

E. El Peligro de la Igualdad Abstracta (==) frente a la Igualdad Estricta (===)

La causa más común de errores en JavaScript radica en el uso del operador ==. Antes de comparar, **== fuerza la conversión de ambos operandos hasta hacerlos compatibles**, lo que produce resultados contradictorios:

```
console.log(0 == false);        // true (false se convierte a 0)
console.log("" == 0);           // true (cadena vacía se convierte a 0)
```

```
console.log("" == false);       // true (ambos se convierten a 0)
console.log(null == undefined); // true (regla especial de ECMAScript)
console.log([] == 0);           // true ([] se transforma en "" y luego en 0)

// EN CAMBIO, CON IDENTIDAD ESTRICTA (===):
// No hay conversión de tipos; exige mismo tipo y mismo valor:
console.log(0 === false);       // false
console.log("" === 0);          // false
console.log(null === undefined); // false
console.log([] === 0);          // false
```

F. Coerción Estricta y Excepciones: Symbol y BigInt

Los tipos primitivos tradicionales (number, string, boolean, null, undefined) permiten conversiones implícitas silenciosas, asumiendo equivalencias matemáticas (como transformar false en 0 o null en 0). Por el contrario, los tipos modernos **Symbol** (ES6) y **BigInt** (ES2020) fueron dotados deliberadamente de **fuertes barreras sintácticas** para impedir fallos de seguridad y pérdida de precisión matemática.

Restricciones de Coerción en Symbol

El tipo Symbol se utiliza para crear identificadores de propiedades de objetos únicos e inmutables. Si el lenguaje permitiese que un Symbol se convirtiese implícitamente en texto, su clave podría quedar expuesta públicamente en logs, cadenas JSON o mensajes de error, destruyendo su propiedad de aislamiento:

```
const idUsuario = Symbol("uid_8841");

// 1. Prohibida la coerción implícita a String con el operador +:
// console.log("Usuario: " + idUsuario);
// ERROR: TypeError: Cannot convert a Symbol value to a string

// 2. Prohibida la coerción matemática a Number:
// console.log(idUsuario * 2);
// console.log(+idUsuario);
// console.log(Number(idUsuario));
// ERROR: TypeError: Cannot convert a Symbol value to a number

// 3. Conversión explícita autorizada:
// Si el programador usa explícitamente String() o .toString(), sí está permitido:
console.log(String(idUsuario));     // "Symbol(uid_8841)"
console.log(idUsuario.toString()); // "Symbol(uid_8841)"

// 4. Conversión a Booleano (permitida tanto implícita como explícitamente):
// Cualquier Symbol se considera siempre un valor TRUTHY:
console.log(Boolean(idUsuario));    // true
if (idUsuario) {                    // Válido: entra al bloque
```

```
  console.log("El símbolo existe en memoria.");
}
```

Restricciones de Coerción en BigInt

El estándar IEEE 754 de los números estándar (number) utiliza 64 bits, limitando los enteros seguros al intervalo $[-(2^{53}-1), 2^{53}-1]$ (Number.MAX_SAFE_INTEGER = $9.007.199.254.740.991$). Superado ese rango, dos números enteros contiguos se redondean al mismo valor binario.

BigInt se diseñó para representar enteros de precisión arbitraria sin desbordamientos. Sin embargo, mezclar implícitamente BigInt con Number ordinario destruiría esa garantía de exactitud, por lo que **la coerción implícita entre ellos está vetada en operaciones matemáticas**.

```
const claveCriptografica = 90071992547409995n; // BigInt (termina
en 'n')
const incremento = 5;                          // Number estándar

// Intento de operación implícita:
// const total = claveCriptografica + incremento;
// ERROR: TypeError: Cannot mix BigInt and other types, use
explicit conversions

// Prohibición con el operador unario positivo (+):
// console.log(+claveCriptografica);
// ERROR: TypeError: Cannot convert a BigInt value to a number
(usa Number() explícito)

// Solución A: Convertir todo a BigInt (preserva precisión entera)
const totalBigInt = claveCriptografica + BigInt(incremento);
console.log(totalBigInt); // 9007199254741000n

// Solución B: Convertir a Number (asumiendo posible pérdida si es
muy grande)
const totalNumber = Number(claveCriptografica) + incremento;
console.log(totalNumber); // 9007199254741000 (como Number
estándar)
```

Coerción en Cadenas de Texto y Booleanos

- **A texto:** Tanto la conversión explícita como la concatenación con el operador + están permitidas; el sufijo n se elimina de la cadena resultante:

  ```
  console.log(500n + " bytes"); // "500 bytes" (coerción
  implícita por presencia de texto)
  console.log(String(123n));     // "123"
  ```

- **A booleano:** Sigue la regla general de valores numéricos:

- 0n es **Falsy**.
- Cualquier BigInt distinto de cero (1n, -10n) es **Truthy**.

```
console.log(Boolean(0n));  // false
console.log(Boolean(15n)); // true
```

G.ACTIVIDADES

Ejercicio 1: El Enigma de la Igualdad con Booleanos

Enunciado: Explica por qué la instrucción if ("0" == false) se cumple con éxito y accede al bloque if, pero en cambio la instrucción if ("0") también se considera verdadera:

```
// Caso A:
if ("0" == false) {
  console.log("A se cumple");
}

// Caso B:
if ("0") {
  console.log("B se cumple");

}
```

Ejercicio 2: ACTIVIDAD PRÁCTICA GUIADA DE LABORATORIO: INSPECTOR DE CONVERSIONES Y TIPADO DÉBIL

Descripción del Escenario

En el desarrollo web en entorno cliente, los datos introducidos por el usuario mediante elementos de formulario (<input>) son interpretados inicialmente por el navegador como cadenas de texto (string). Asimismo, las respuestas JSON obtenidas desde servicios backend pueden contener valores booleanos, numéricos, nulos o matrices serializadas. Comprender cómo reacciona el motor ECMAScript ante conversiones explícitas (Number(), String(), Boolean()) e implícitas (mediante los operadores + y -) es crítico para prevenir fallos silenciosos en la lógica de negocio.

En esta práctica, se proporciona la interfaz visual completa (laboratorio_conversiones.html). Tu tarea como desarrollador front-end consiste en programar la lógica completa del archivo externo laboratorio_conversiones.js partiendo desde cero.

Material Entregado: Estructura HTML y CSS (**laboratorio_conversiones.html y estilos.css**)

Crea los archivos laboratorio_conversiones.html y estilos.css y pega el código correspondiente. **No debes modificar estos documentos**; todo el comportamiento debe implementarse de forma desacoplada en el archivo JavaScript correspondiente.

`estilos.css` **(Hoja de estilos externa)**

```
body {
  font-family: monospace;
  margin: 30px;
  background-color: #1e1e2e;
  color: #cdd6f4;
}

.panel {
  max-width: 600px;
  background: #313244;
  padding: 25px;
  border-radius: 8px;
  border: 1px solid #45475a;
}

h2 {
  color: #89b4fa;
  margin-top: 0;
}

.campo {
  margin-bottom: 15px;
}

label {
  display: block;
```

```
  margin-bottom: 5px;
  color: #a6adc8;
}

input[type="text"] {
  width: 95%;
  padding: 8px;
  background: #181825;
  border: 1px solid #585b70;
  color: #fab387;
  font-size: 14px;
}

button {
  padding: 8px 16px;
  background: #89b4fa;
  color: #11111b;
  border: none;
  font-weight: bold;
  cursor: pointer;
  border-radius: 4px;
}

button:hover {
  background: #b4befe;
}
```

```
.resultados {
  margin-top: 20px;
  padding: 15px;
  background: #181825;
  border-left: 4px solid #89b4fa;
  line-height: 1.6;
}

.valor-resaltado {
  color: #a6e3a1;
  font-weight: bold;
}

.alerta {
  color: #f38ba8;
}
```

2. laboratorio_conversiones.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Laboratorio CE 2.d: Conversión de Tipos</title>
  <!-- Vinculación del CSS externo -->
  <link rel="stylesheet" href="estilos.css">
</head>
```

```
<body>


  <div class="panel">

    <h2>Laboratorio CE 2.d: Inspector de Coerción</h2>


    <div class="campo">

      <label for="txtEntrada">Valor literal a inspeccionar (ej: 0,
"0", false, null, undefined, []):</label>

      <input type="text" id="txtEntrada" value="0">

    </div>


    <button id="btnAnalizar">Inspeccionar Conversiones</button>


    <div id="pantallaSalida" class="resultados">Esperando
datos...</div>

  </div>


  <!-- Vinculación del script externo -->

  <script src="laboratorio_conversiones.js"></script>

</body>

</html>
```

Requisitos Técnicos y Funcionales de la Práctica

Debes crear el archivo laboratorio_conversiones.js (puedes ayudarte de los archivos de laboratorios dados en prácticas anteriores y reutilizar código) en el mismo directorio que el archivo HTML y estructurar el código cumpliendo los siguientes hitos técnicos:

Hito 1: Captura de Eventos y Normalización del Dato de Entrada

- Localiza mediante el DOM los elementos btnAnalizar, txtEntrada y pantallaSalida.

- Asocia al botón un escuchador de eventos click que invoque a una función controladora principal (analizarCoercion).
- **Tratamiento del texto de entrada:** El valor del campo de texto llega siempre como un string. Para poder evaluar primitivos reales (como números literales, booleanos, matrices u objetos), procesa el texto mediante un bloque try...catch con JSON.parse().
    - Si JSON.parse() tiene éxito, utiliza el valor transformado resultante.
    - Si falla, gestiona manualmente los literales que no son JSON válido: si el texto es "undefined", asigna el valor primitivo undefined; si es "NaN", asigna el valor NaN; en cualquier otro caso, conserva la cadena literal tal cual.

```
// Normalización de la entrada literal a tipo primitivo o de referencia real

  let valorReal;

  try {

    valorReal = JSON.parse(textoEntrada);

  } catch (error) {

    // Casos especiales que no son JSON válido pero representan primitivos en JS:

    if (textoEntrada === "undefined") {

      valorReal = undefined;

    } else if (textoEntrada === "NaN") {

      valorReal = NaN;

    } else {

      valorReal = textoEntrada; // Cadena alfanumérica pura

    }

  }
```

Hito 2: Conversión Explícita y Diagnóstico de Tipos

- Aplica las funciones constructoras globales para obtener las tres representaciones del dato analizado:
    1. Numérica: Number(valorReal).
    2. Cadena: String(valorReal).

3. Booleana: Boolean(valorReal).

- Inspecciona el tipo primitivo del dato original mediante el operador typeof.

Hito 3: Detección de Anomalías de Coerción

- Comprueba si el texto introducido es "0" y evalúa si su valor booleano resultante es true. Si es así, prepara un mensaje con la clase .alerta explicando por qué una cadena que contiene un cero es un valor *truthy*.
- Comprueba si el texto es "[]" y evalúa si su conversión numérica resulta en 0. Prepara una advertencia indicando que una matriz vacía se coerciona al número cero.

Hito 4: Prueba en Vivo de Suma vs. Resta

- Realiza las operaciones valorReal + 10 y valorReal - 10.
- Inspecciona el tipo resultante de cada una (typeof) para evidenciar en pantalla si el operador + forzó una concatenación textual o una adición matemática, contrastándolo con el forzado puramente numérico del operador -.

Hito 5: Control Estricto con Number.isNaN()

- Evalúa si la conversión numérica Number(valorReal) produce un valor no computable utilizando el método estricto Number.isNaN().
- Si se evalúa a true, añade al informe de salida un aviso destacado en rojo (.alerta) indicando: "Conversión numérica fallida: No es un número válido".

Hito 6: Inyección Estructurada en el DOM

- Vuelca toda la información en el contenedor #pantallaSalida utilizando formato HTML con etiquetas , con clase .valor-resaltado y saltos de línea
.

  Como Número [Number(x)]: ${comoNumero} (typeof: ${typeof comoNumero})

Guía de Casos de Prueba para la Verificación

Antes de entregar tu práctica, abre la aplicación en el navegador y comprueba que arroja las siguientes salidas para cada valor de prueba:

Valor a introducir en #txtEntrada	Number(x) esperado	Boolean(x) esperado	Comportamiento x + 10 vs x - 10	Alertas que deben saltar
0	0	false	10 / -10	Ninguna

"0" (con comillas)	0	**true**	"010" (concatena) / -10 (resta)	Aviso: "0" es TRUTHY
false	0	false	10 (false → 1) / -10	Ninguna
null	0	false	10 (null → 0) / -10	Ninguna
undefined	NaN	false	NaN / NaN	Alerta Number.isNaN()
[]	0	**true**	"10" (concatena) / -10	Aviso: Matriz coerciona a 0
"texto"	NaN	true	"texto10" / NaN	Alerta Number.isNaN()

Rúbrica de Evaluación Específica (CE 2.d)

Criterio Específico	**Nivel Insuficiente (0 - 4.9)**	**Nivel Suficiente / Bien (5 - 8.9)**	**Nivel Excelente (9 - 10)**	**Ponderación**
Captura y normalización segura del dato	No extrae el dato o la aplicación se congela al introducir objetos/matrices.	Usa JSON.parse pero no gestiona adecuadamente cadenas no JSON ni undefined/NaN.	Normaliza con bloque try...catch solventando cualquier literal complejo sin errores en consola.	25%

Aplicación de conversiones explícitas	Faltan métodos de conversión o confunde typeof con el tipo de dato.	Realiza las tres conversiones básicas (Number, String, Boolean) pero no reporta sus tipos con typeof.	Realiza y muestra con rigor las conversiones numéricas, de texto y booleanas junto a su tipado estricto.	25%
Comprobación de peculiaridades (+ vs -)	No evalúa la sobrecarga del operador suma ni la diferencia con la resta.	Realiza los cálculos pero no muestra los tipos resultantes para evidenciar la coerción implícita.	Evidencia y formatea con claridad la divergencia entre concatenación y adición aritmética.	25%
Detección de anomalías y validación de NaN	Omite las comprobaciones de valores especiales o no usa Number.isNaN().	Detecta NaN mediante comparaciones erróneas (x === NaN) o solo cubre una anomalía.	Aplica Number.isNaN() correctamente y emite alertas didácticas para "0" y [].	25%